杭州电子科技大学

《计算机网络实验》实验报告

课程号：(2023-2024-1)-S0507170-09

实验五　UDP 抓包及分析

指导老师：

姓名学号：

2026 年 11 月

一、实验目的

- 掌握用捕获过滤器与显示过滤器筛选 UDP 报文的方法；
- 通过实际报文逐字段解读 UDP 首部的四个字段，理解各字段的长度与取值范围；
- 验证 UDP 长度字段的含义，并据此推导 UDP 载荷的理论上限；
- 在 IP 首部中找到协议号字段，确认 UDP 的协议号 17（0x11）；
- 观察一对请求/应答报文的端口互换关系，以及 UDP 无连接、不重传带来的后果。

二、实验设备和环境

表 1 实验软硬件环境

项目	配置说明
客户端主机	Windows 11，IPv4 192.168.1.3（WLAN 网卡）
对端	默认网关兼 DNS 服务器 192.168.1.1，端口 53
产生流量的方式	nslookup 查询域名（DNS 基于 UDP），再手工发送一个畸形 UDP 报文作对照
抓包工具	Wireshark 3.6.16 / dumpcap
捕获过滤器	udp
捕获文件	cap_udp.pcapng（21 个报文，其中 7 个属于实验主动产生的流量）

UDP 本身不产生"可主动访问的服务"，因此本实验用 DNS 查询作为 UDP 流量的来源：DNS 是典型的"一问一答"式 UDP 应用，报文短小、字段清晰，非常适合用来观察 UDP 首部。另外本机后台还有联想的更新检查等程序在发 DNS 查询，这些报文是真实网络环境的一部分，分析时通过源端口和查询名即可把它们与主动产生的流量区分开。

三、实验过程及步骤

3.1 抓包

因为 UDP 报文极短且转瞬即逝，捕获过滤器只写 udp 即可，不必限定端口：

```
> "E:\tools\Wireshark\dumpcap.exe" -i 6 -f "udp" -a duration:30 -w cap_udp.pcapng
```

抓包期间执行下面的查询，让本机产生 UDP 流量：

```
> nslookup www.hdu.edu.cn
服务器:  UnKnown
Address:  192.168.1.1

非权威应答:
名称:    www.hdu.edu.cn
Addresses:  218.75.123.179
```

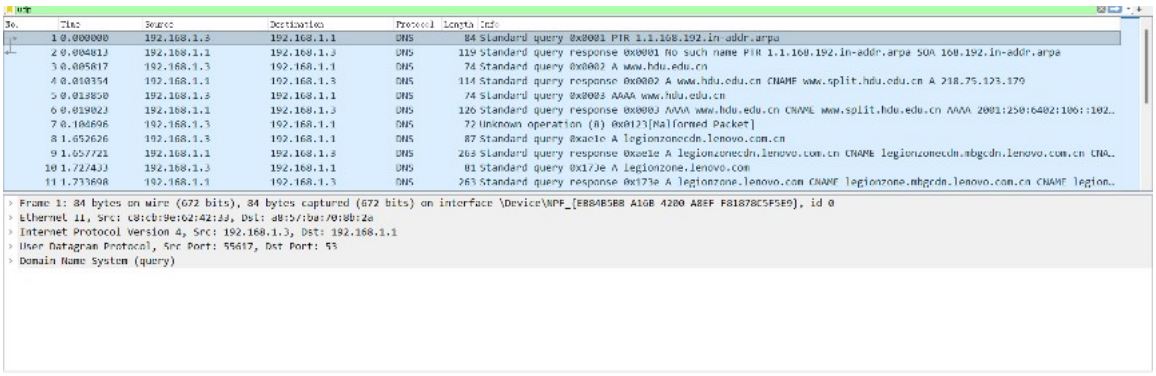


图 1 显示过滤器 udp 筛选后的报文列表（共 21 个，全部为 DNS 载荷）

列表中 Protocol 列显示的是 DNS 而不是 UDP，这是因为 Wireshark 会按端口号把载荷进一步解码到应用层；要看 UDP 首部本身，必须展开详情窗格的 User Datagram Protocol 层。另外要注意 Length 列显示的是整个帧的长度（例如 84 字节），并不是 UDP 首部里的 Length 字段。

3.2 UDP 首部的四个字段

选中第 3 号报文（本机发往 192.168.1.1 的 A 记录查询），展开 User Datagram Protocol 层，可以看到四个字段（图 2）。

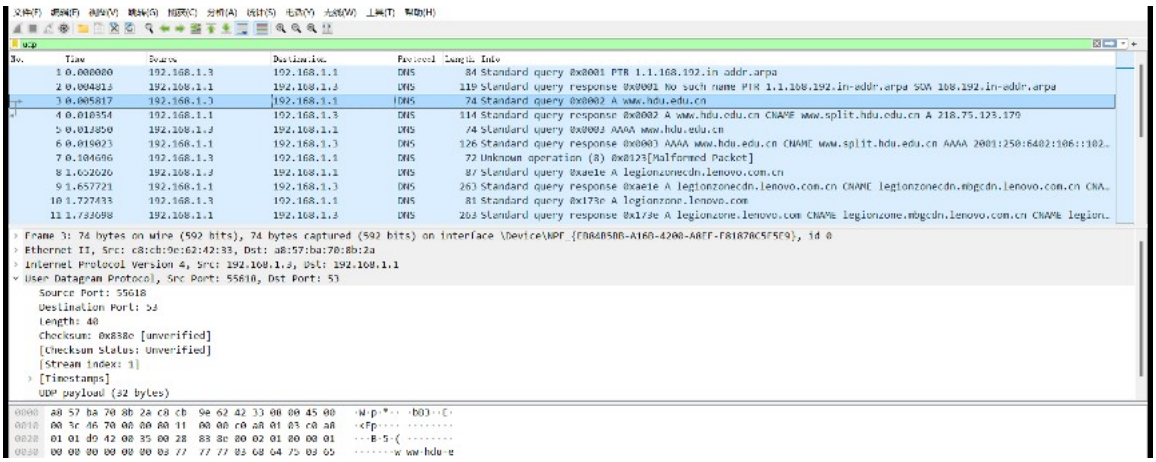


图 2 UDP 首部四个字段：源端口 55618、目的端口 53、长度 40、校验和 0x838e

表 2 UDP 首部字段

字段	字节数	本报文取值	含义
Source Port（源端口）	2	55618	发送方进程的端口，由操作系统从临时端口范围中随机分配
Destination Port（目的端口）	2	53	接收方进程的端口，53 是 DNS 服务的知名端口
Length（长度）	2	40	UDP 首部加 UDP 载荷的总字节数，本报文即 8 + 32 = 40
Checksum（校验和）	2	0x838e	对首部与载荷（含伪首部）计算的 16 位反码和，用于差错检测

四个字段各占 2 字节，合计 8 字节，这是 UDP 首部的固定长度，也是 UDP 与 TCP 最直观的差别之一：TCP 首部最少 20 字节，而 UDP 只有 8 字节。Wireshark 在字段下方还给出了 UDP payload (32 bytes) 与 [Stream index: 1]，32 字节正好等于 Length 40 减去首部 8 字节，数据自洽。

3.3 校验和为何显示 unverified

截图里 Checksum 一行后面跟着 [unverified] 与 [Checksum Status: Unverified]，这常常被误解为"校验和错误"。实际上这是网卡校验和卸载（checksum offload）造成的：本机的网卡驱动在把报文交给 Wireshark 抓包时还没有填入真实的校验和，真正的校验和是在网卡硬件里计算并写入的。因此抓包文件里的这个值可能为 0 或任意值，Wireshark 无法据此判断对错，只能标注为"未验证"。要真正验证校验和，需要在 Wireshark 的协议首选项里关闭 UDP checksum validation，或者在一台不做卸载的机器上抓包。

3.4 IP 首部中的协议号

UDP 首部的上一层是 IP 首部。展开 Internet Protocol Version 4 层，最后一行 Protocol 显示为 UDP (17)（图 3）。

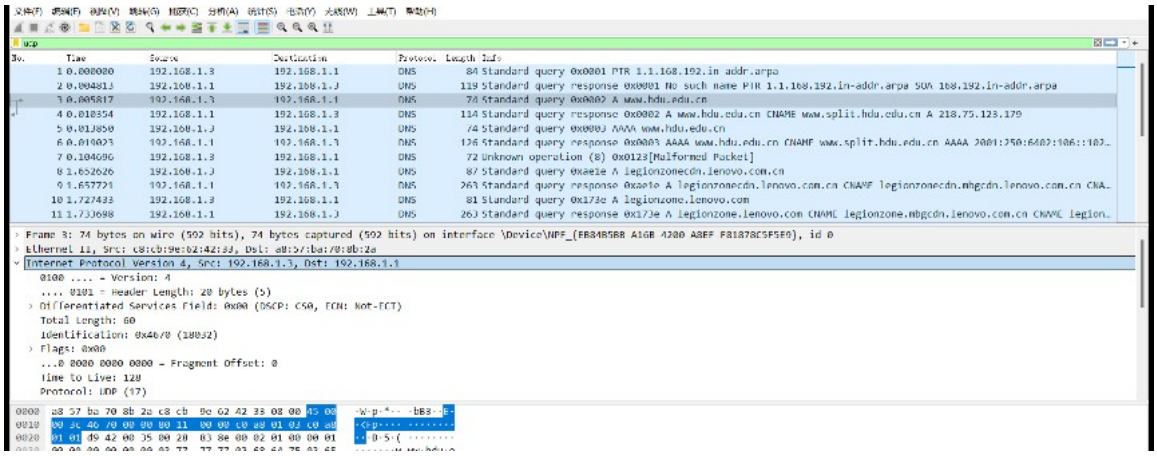


图 3 IP 首部中的 Protocol 字段：UDP (17)，即十六进制 0x11

这个字段是 IP 层用来告诉接收方"载荷应该交给哪个上层协议"的标识。17 是 IANA 为 UDP 分配的正式编号，换算成十六进制是 0x11。作为对照，TCP 的协议号是 6，ICMP 是 1。在十六进制窗格里也能直接看到这个字节：以太网首部之后第 10 个字节即 IP 首部的第 9 个字节，值为 11（十六进制）。

3.5 请求与应答的端口互换

第 3 号报文与第 4 号报文构成一对请求/应答（图 4）。

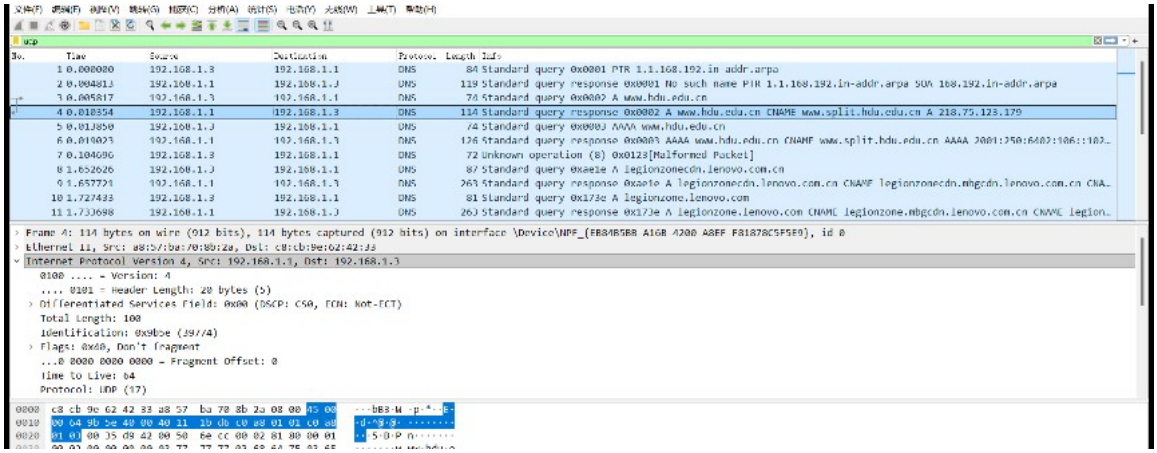


图 4 应答报文的 UDP 首部：源端口 53、目的端口 55618

表 3 一对请求/应答报文的端口对应关系

报文	方向	源端口	目的端口	Length
第 3 号（请求）	192.168.1.3 → 192.168.1.1	55618	53	40
第 4 号（应答）	192.168.1.1 → 192.168.1.3	53	55618	80

可以看到两个端口恰好互换：请求的目的端口（53）成为应答的源端口，请求的源端口（55618）成为应答的目的端口。这正是 UDP（以及 TCP）多路复用的基础：服务器固定监听知名端口，而客户端每次用一个新的临时端口，服务器把应答发回这个临时端口，操作系统再根据端口号把数据交给对应的进程。如果本机同时打开了多个 DNS 查询，它们会用不同的临时端口（本实验中依次是 55617、55618、55619、55620），这样应答就不会串到别的进程去。

3.6 UDP 无连接、不重传的直接证据

为了验证 UDP 的"尽力而为"特性，本实验用 Python 手工构造并发送了一个残缺的 DNS 报文：只填了 12 字节的 DNS 首部，事务 ID 取 0x0123、标志位填了一个非法的 0x4567，后面的问题区完全没有。抓包中它是第 7 号报文（图 1 列表中的 "Unknown operation (8) 0x0123 [Malformed Packet]"）。

```
import socket, struct
payload = struct.pack('!HHHHHH', 0x0123, 0x4567, 1, 0, 0, 0)
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.sendto(payload, ('192.168.1.1', 53))   # 发出后立即返回，不等回复
s.close()
```

这个报文发出后，抓包文件里没有任何对应的应答，也没有任何重传。对比一下：如果这是 TCP，发送方在超时后会重发同一个报文，接收方也会回一个 RST 告诉对端"这个请求我看不懂"。UDP 什么都没有做——它把报文交给 IP 层就结束了，报文是否到达、对端是否理解，UDP 一概不管。DNS 服务器收到了这个畸形报文后直接丢弃，这也是 DNS 协议自身的健壮性设计：格式不对就不回复，避免被放大攻击利用。

同样地，第 3 号报文与第 4 号报文之间的时间差只有 4.5 ms，而这个 4.5 ms 里没有任何确认、握手或窗口协商，一问一答就把事情办完了。这种极低的开销正是 DNS、DHCP、音视频流、游戏同步等应用选择 UDP 的原因。

3.7 UDP 长度字段的取值范围

Length 字段占 2 字节，取值范围是 0 到 65535。由于它已经包含了 8 字节的首部，因此单个 UDP 数据报能承载的载荷最多是 65535 − 8 = 65527 字节。但这只是协议层面的理论上限，实际可用的最大值还要受 IP 层与链路层限制：

表 4 不同层次对 UDP 载荷的限制

层级	限制来源	最大 UDP 载荷
UDP 协议	Length 字段 16 位	65535 − 8 = 65527 字节
IPv4	Total Length 字段 16 位，减去 20 字节 IP 首部	65535 − 20 − 8 = 65507 字节
以太网	MTU 1500，需 IP 分片	1500 − 20 − 8 = 1472 字节（不分片时）

一旦超过 MTU，IP 层就要分片；而 UDP 报文只要丢一个分片，整个数据报就作废，所以实际应用中通常会把 UDP 载荷控制在 1472 字节以内，或者干脆改用 TCP。本实验中最大的 UDP 长度是第 9 号报文的 263 字节，远小于 MTU，因此不存在分片问题。

四、实验问题解答

表 5 UDP 实验问题逐条解答

序号	问题	解答与依据
1	UDP 首部有几个字段？分别叫什么？	4 个字段：Source Port（源端口）、Destination Port（目的端口）、Length（长度）、Checksum（校验和）。依据是第 3 号报文展开 User Datagram Protocol 层后恰好只有这四行，没有序号、确认号、标志位、窗口等 TCP 才有的字段。
2	每个 UDP 首部字段各占多少字节？	各占 2 字节，合计 8 字节。依据：第 3 号报文的 Length 字段值为 40，而 Wireshark 显示 UDP payload 为 32 字节，40 − 32 = 8，正好是首部长度；再用十六进制窗格核对，UDP 首部在以太网（14 字节）+ IP（20 字节）之后，恰好占用 8 个字节即 4 个 16 位字。
3	Length 字段的值是什么的长度？用抓到的报文验证。	是 UDP 首部加 UDP 载荷的总长度（不含 IP 首部）。验证：第 3 号报文 Length=40，其 UDP payload 为 32 字节，40 = 8 + 32；第 4 号报文 Length=80，其 UDP payload 为 72 字节，80 = 8 + 72。两者的 IP 首部都是 20 字节，但 Length 字段都没有把 IP 首部算进去。
4	UDP 载荷最多能有多少字节？	65535 − 8 = 65527 字节。推理过程：Length 字段 16 位，最大值 65535，而该字段包含 8 字节首部，故载荷上限为 65527。但这只是 UDP 协议层上限；受 IPv4 的 Total Length（16 位，

序号	问题	解答与依据
		含 20 字节 IP 首部）限制，实际最多 65507 字节；在以太网上不分片时则为 1472 字节。
5	源端口的最大值是多少？	65535。源端口与目的端口都是 16 位无符号整数，取值范围 0–65535。其中 0–1023 是知名端口（需要管理员权限才能绑定），1024–49151 是注册端口，49152–65535 是临时端口。本实验客户端使用的是 55617–55620 等临时端口。
6	UDP 的协议号是多少（十六进制与十进制）？	十六进制 0x11，十进制 17。依据：展开 IP 首部后 Protocol 字段显示为 "UDP (17)"，在十六进制窗格中对应字节的值为 11。这个字段位于 IP 首部第 9 个字节，接收方据此把载荷交给 UDP 模块处理。
7	一对请求/应答报文中，两个报文的端口号有什么关系？	恰好互换。以第 3、4 号报文为例：请求的源端口 55618、目的端口 53，应答的源端口 53、目的端口 55618。即"请求的目的端口 = 应答的源端口"，"请求的源端口 = 应答的目的端口"。这体现了 UDP 的多路复用机制：服务器固定在知名端口监听，客户端使用一次性临时端口，应答按端口号回到发起查询的那个进程。

五、实验总结

5.1 遇到的问题与解决办法

表 6 UDP 实验中的常见问题与处理

问题现象	原因分析	解决办法
列表里 Protocol 列显示 DNS 而不是 UDP，找不到 UDP 首部	Wireshark 按端口把载荷解码到了应用层	在详情窗格展开 User Datagram Protocol 层即可，或用显示过滤器 udp 后右键 → 解码为 → UDP
Checksum 显示 [unverified] 或 0x0000	网卡校验和卸载，抓包时真实校验和尚未写入	这是正常现象，在报告中说明；如需真实验证可关闭 UDP 校验和验证或换机器抓包
后台程序产生的 UDP 报文混在结果里	联想更新程序、wpad 探测等也在发 DNS 查询	按源端口（55617–55620 为本次主动产生）或按 dns.qry.name 精确筛选
手工构造的畸形 DNS 报文没有任何回复	报文格式非法，服务器直接丢弃；UDP 本身也不重传	这正是实验要证明的现象，在报告中作为"UDP 无连接、不重传"的证据使用

5.2 收获与体会

UDP 大概是整个课程里最简单的协议，只有四个字段、八个月字节，但它恰恰因为"什么都不做"而变得有用。这次实验最有价值的不是记住那四个字段的名字，而是亲手制造了一个错误报文，然后看着它石沉大海：没有应答、没有重传、也没有 RST。如果换成 TCP，同样的错误报文会立刻换来一个 RST，或者发送方在超时后重发。这种"发出去

就不管"的对比，比任何文字描述都更能说明 UDP 的定位——它不是在 TCP 上偷工减料，而是主动放弃了 TCP 提供的那些保证，把可靠性留给应用层自己决定。

第二个收获是对"长度"这一概念的分层理解。同一个数据报，在 UDP 层看到的长度是 40 字节，在 IP 层是 60 字节，在以太网帧里是 74 字节。一开始我以为是抓包工具算错了，把三个数字逐个减去各层首部才理清：UDP 的 Length 只管自己，IP 的 Total Length 管到 IP 载荷，而 Wireshark 列表里的 Length 列显示的是整个帧。这提醒我读协议字段时必须先确认"这个长度是从哪里算到哪里"，否则很容易得出"报文对不上"的错误结论。

第三个收获是对协议号与端口的分工有了清晰认识。协议号在 IP 首部里，告诉主机"这是给哪个传输层协议"；端口在传输层首部里，告诉主机"这是给哪个进程"。两者是两级分用：先按协议号分到 UDP 模块，再按端口号分到具体进程。这个两级结构在配置防火墙时特别重要——只封端口而不封协议号，或者只封协议号而不封端口，效果是完全不同的。